MemMux: Runtime Verification and Honest Resource Attribution for Fleets of Parallel Coding Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

Developers increasingly run not one coding agent but a fleet of them side by side on a single workstation. The tools they reach for—terminal multiplexers like `tmux` and a new generation of agent managers—were built to arrange windows, not to govern memory. When ten agents each spawn language servers, test runners, and browsers, nothing on the machine can say how much memory belongs to which agent, guarantee that a terminated agent's descendants are actually gone, notice a child that has escaped its agent, or keep the machine off the swap cliff without an OOM kill silently discarding uncommitted work. We argue these are *verification* problems: an agent-hosting substrate should continuously emit signals an operator (or an auditor) can check. We present MemMux, a local runtime that treats resource governance as a set of checkable guarantees—honest per-agent attribution, complete reclamation, escaped-process visibility, bounded footprint under overcommit, and low monitoring overhead—and a claims-disciplined benchmark that measures each guarantee against `tmux`, a purpose-built agent multiplexer, and a raw-process baseline running identical workloads. On a 32 GB Linux reference host, as the fleet grows to ten agents, MemMux holds peak footprint flat at 435 MiB while the ungoverned tools grow linearly to 2.1 GB; it reclaims 100% of a terminated agent's process subtree where the raw baseline strands half of it (ten orphaned processes, 303 MiB, at N=10); and it is the only system that surfaces an escaped child, detecting 10/10 injected escapes versus a capability the others structurally lack. We report an honest cost: the always-on 1 Hz attribution scan stays near 0.6% CPU for a single agent but reaches 2.7% at ten, over our 2% target. Driving the same harness with real Claude Code sessions confirms that 100% attribution and low overhead carry over from the stub to live, multi-process agent trees. We release the engine, the benchmark, and a one-command reproducer.

1 Introduction

A year ago, running an autonomous coding agent meant babysitting a single session. Today it is normal to fan a task out across several agents at once—one refactoring, one writing tests, one chasing a flaky build—and to watch them in a grid of panes. The agents themselves have improved quickly [Yao et al., 2023, Shinn et al., 2023, Jimenez et al., 2024, Yang et al., 2024]; the substrate they run on has not. Developers arrange these fleets with terminal multiplexers such as `tmux` [Marriott and tmux contributors, 2024] or with purpose-built

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

agent managers, and both were designed to place windows on a screen, not to answer questions about memory.

Those questions matter because a coding agent is not a single process. It is a tree: the model's CLI, a shell, language servers, test and build workers, sometimes a headless browser. A handful of such trees can exhaust a laptop's RAM. When they do, the operating system's response—the Linux OOM killer, or swap thrashing—is blunt and invisible: a process is killed mid-write, or the whole machine grinds, and the agent's uncommitted edits are simply gone. Nothing in the stack was accountable for that memory in the first place. `tmux` does not know that pane 7's language server belongs to the same logical agent as pane 3's test runner; it cannot tell you that a double-forked child has escaped supervision; and it has no budget it is trying to keep.

We think the right lens for this is *runtime verification* [Bartocci et al., 2018]: rather than prove properties of the substrate ahead of time, instrument it so the properties it claims can be *checked while it runs*, and surface any violation rather than hide it. Cluster schedulers made an analogous move a decade ago—Borg combined admission control, over-commitment, and process-level isolation, and treated monitoring as a first-class output [Verma et al., 2015]. We bring that stance down to a single developer machine and to the specific failure modes of agent fleets.

This paper contributes: (i) **five verification signals for agent fleets** (Section 4)—honest per-agent attribution, complete reclamation on teardown, escaped-process visibility, bounded footprint (and preserved work) under overcommit, and bounded monitoring overhead—each a monitor with a stated threshold; (ii) **MemMux** (Section 3), a local runtime that enforces them; and (iii) **a claims-disciplined benchmark** (Section 4) that drives `tmux`, an agent multiplexer, and a raw baseline through *identical* deterministic workloads, tags every process as agent or manager overhead, and never reports a number it did not measure—where a competitor cannot be driven honestly, it is recorded as skipped, not estimated. On a 32 GB Linux reference host the guarantees hold and, under overcommit, separate MemMux from the ungoverned tools by $\sim 5\times$ at ten agents (Section 5). We are candid about scope: the workload is a deterministic stub rather than live models, the fleets are modest ($N \leq 10$), and the monitoring cost is not yet under 2%—all discussed in Section 6.

2 Background and Related Work

Language-model agents. Interleaving reasoning with tool use [Yao et al., 2023] and adding self-reflection [Shinn et al., 2023] turned language models into agents that take long sequences of actions. Coding is the canonical proving ground: SWE-bench frames the task as resolving real GitHub issues against a repository [Jimenez et al., 2024], and SWE-agent shows a well-designed agent–computer interface matters as much as the model [Yang et al., 2024]. Multi-agent settings, first studied for open-ended behavior [Park et al., 2023], are now a practical reality on developer machines. This work is orthogonal to the agents: we govern whatever they spawn.

Terminal multiplexers. `tmux` [Marriott and tmux contributors, 2024] is the default way developers run many long-lived sessions at once. It is excellent at persistent panes and detach/reattach—the honest baseline any "run many agents" tool must beat—but a pane is just a process group under a server with no notion of a logical agent, a memory budget, or a process that has wandered out of its subtree.

Resource management. Cluster managers solved fleet-scale governance long ago: Borg with admission control and over-commitment [Verma et al., 2015], Mesos with two-level scheduling [Hindman et al., 2011], and Kubernetes as the open-source lineage [Burns et al., 2016]. The enforcement primitives they use—Linux control groups [Linux Kernel Documentation, 2024a] and the OOM killer [Linux Kernel Documentation, 2024b]—live in every modern kernel. MemMux borrows the ideas (reserve, admit, reclaim before you thrash) but targets a single un-orchestrated laptop, and it deliberately avoids hard cgroup containment in favor of signal-based supervision so it runs unprivileged.

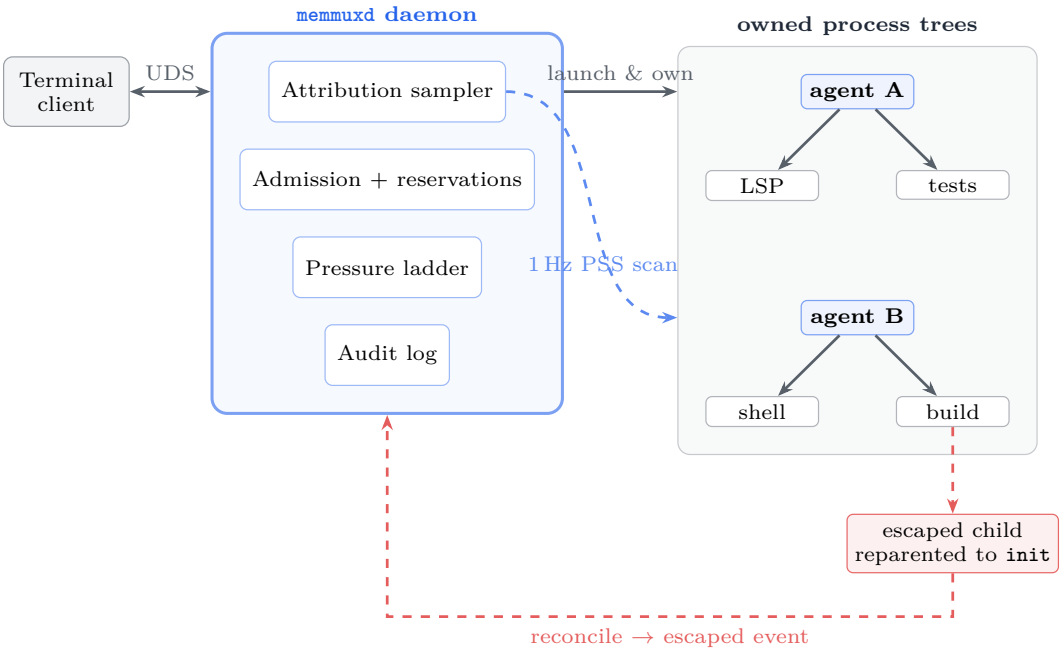

Figure 1: MemMux architecture. A terminal client drives the `memmuxd` daemon over a Unix domain socket. The daemon launches and *owns* each agent's process tree; a 1 Hz sampler sums proportional memory (PSS) across every owned subtree (H1), reconciliation flags any child that double-forks out to `init` (H3), and admission, the pressure ladder, and an audit log turn the budget into checkable, logged decisions (H2/H4/H5).

Memory accounting. Summing resident set size (RSS) over a process tree double-counts shared pages. The proportional set size (PSS) exposed through `/proc/<pid>/smaps_rollup` divides each shared page by its sharers and is the honest per-process figure [Linux Kernel Documentation, 2024c]; macOS offers `phys_footprint` as its analogue. MemMux attributes by PSS (RSS fallback) so a task's number reflects memory it is responsible for.

Runtime verification. Rather than verify a system statically, runtime verification instruments it and checks a specification against the execution trace, reacting to violations [Bartocci et al., 2018]. Our five signals are exactly such monitors: cheap, always-on checks with explicit thresholds whose failures are events, not silence.

3 The Design of MemMux

MemMux is a Rust workspace: a daemon (`memmuxd`) that owns state and does the governing, and a terminal client that attaches over a Unix domain socket. A *task* is a durable, logical agent; a *runtime* is one physical launch of a provider inside a pseudo-terminal. The design is organized around making each signal checkable (Figure 1).

Attribution. Once per second the daemon snapshots the host process tree and, for each running task's provider root, sums the PSS of the root and all descendants. That per-task figure feeds the budget, the client's task view, and—because we record which pids were seen under each root—the escaped-process check. Attribution is scoped to processes MemMux launched, so unrelated host activity never pollutes a task's number.

Recursive reclamation. When a task is terminated (by the operator or because its provider exited), MemMux reaps the *entire* owned subtree. The order is load-bearing: the subtree is signalled while the root is alive, because once the root dies its children reparent to `init` and the tree that identifies them is lost. For a provider that already exited, MemMux

	Signal	Monitor / threshold
H1	Attribution	$\geq$ 95% of a launched tree's proportional memory mapped to its owning task
H2	Cleanup	$\geq$ 99.5% of an owned subtree gone $\leq$ 10 s after the launcher's normal teardown
H3	Escaped visibility	every process that escapes its agent is surfaced as an event
H4	Bounded / no-loss	footprint held near budget under overcommit; uncommitted work checkpointed before any reclaim
H5	Overhead	continuous monitoring costs $\leq$ 2% CPU at fleet scale

Table 1: The five runtime-verification signals and their pass thresholds.

reaps from the pids recorded while the task ran. The target is $\geq$ 99.5% of owned descendants gone within ten seconds.

Escaped-process reconciliation. A child that double-forks and reparents to `init` has escaped its agent—the classic way a leak hides. Each sample, MemMux reconciles the pids seen under a task against the live tree; a once-owned pid now living outside every task subtree is flagged *escaped* and surfaced as an event. Reconciliation is scoped to the managed set, so the signal is a real "we lost track of a process we started," not host noise.

Admission and the pressure ladder. The daemon holds a memory budget (physical RAM minus reserves for the OS, interactive apps, and a swap-safety margin). Starting a task reserves its predicted peak; if the reservation would exceed the budget the task stays queued with a visible reason rather than over-subscribing the machine. When live usage climbs anyway, a graduated *pressure ladder* responds before the machine thrashes: reclaim escaped children, hibernate the lowest-priority idle task, recycle the largest provider, and—only at the top, and only after persisting a checkpoint of the victim's Git state—terminate the lowest-priority tree. Preserving work always precedes destroying a process. Every governance action writes an audit decision with its reason and evidence, which is what makes the system verifiable rather than merely automatic.

4 Verification Signals and Methodology

We frame governance as five monitors, each with a threshold (Table 1). A benchmark harness drives every system under test through identical, deterministic workloads and evaluates the monitors from an external process sampler—the same accounting code the daemon uses, pointed at whatever root pids a launcher reports, so measurement is product-agnostic.

Launchers. Each launcher starts N identical agents and reports the pids that constitute the fleet: a *raw* baseline (spawn the stub directly), `tmux` (one pane per agent on an isolated server), and MemMux (create N tasks through the real daemon). A fourth, the `herdr` agent multiplexer, is driven through its socket API where installed; on our stock Linux reference host it was absent and is reported as skipped rather than guessed, as are `dmux` (which requires a controlling terminal and cannot be driven headlessly), `cmux`, and `agentmux`.

Workload. A deterministic stub agent replays a recording of allocate / spawn-child / emit / sleep steps, touching every page it allocates so its resident memory is real. Using a stub rather than a live model is a deliberate choice: it makes runs reproducible and identical across launchers, at the cost of realism (Section 6). A *hold* scenario keeps N agents, each with a live child, concurrently resident for the measurement window; without it, short workloads finish before a fleet can be observed—an error we hit and corrected early.

Claims discipline. The harness records each launcher's version, the measurement date, and the host; it tags every sampled process as agent or manager overhead so the two are reported separately; and it emits a number only for a monitor it actually measured. A

Launcher	N=1	N=5	N=10
raw baseline	215	1074	2146
`tmux` 3.4	218	1076	2149
MemMux 0.8.0	220	**435**	**435**

Table 2: Peak fleet footprint (MiB) under a 3000 MiB budget (*overcommit*). The ungoverned launchers scale linearly with the fleet (raw and `tmux` track each other to within noise); MemMux holds flat at 435 MiB by deferring admissions—a 4.9× separation at N=10.

launcher that cannot be driven contributes "unsupported," never a fabricated zero. These rules are as much a contribution as the engine: fair comparison of agent-hosting tools has no accepted methodology, and an unfair one is worse than none.

5 Evaluation

Setup. The reference host is an AWS `m7i.2xlarge`: 8 vCPUs (Intel Xeon Platinum 8488C), ≈31 GiB RAM, an 8 GiB swap file, Ubuntu 24.04 (Linux 6.17). Versions: `tmux` 3.4, MemMux 0.8.0. Each cell is two trials; we report mean ± 95% CI. We swept fleet sizes $N \in \{1, 5, 10\}$ across three scenarios. A macOS cross-check (Apple M4 Max, 32 GiB), which additionally exercised `herdr`, agreed qualitatively: `herdr`, like the other ungoverned managers, reaped its subtrees on teardown (100% cleanup) but detected no escapes and grew its footprint with the fleet.

5.1 H4: bounded footprint under overcommit

We give MemMux a budget of 3000 MiB—below the aggregate reservation of the larger fleets—and start all N agents under each launcher. Table 2 reports peak total footprint against fleet size. The ungoverned launchers scale linearly: at ten agents they reach 2.1 GB. MemMux admits what fits (one agent at N=1; two at N=5 and N=10, deferring the rest with a visible reason—3 and 8 deferrals respectively) and holds peak footprint flat at 435 MiB, a 4.9× separation at N=10. This is the sharpest result in the paper and the one that most directly motivates governance: without a budget, a fleet's footprint is simply the sum of its agents.

The *no-lost-work* half of H4 is a capability we verify but did not trigger here: because MemMux defers at admission, the pressure ladder rarely needs to reclaim a running victim. A daemon-level test confirms the guarantee directly—a task with a dirty Git worktree, forced into the emergency stage, produces a checkpoint carrying its patch hash *before* the process is terminated. Ungoverned launchers have no checkpoint mechanism, so this is reported as a capability they lack, not a number.

5.2 H2: complete cleanup on teardown

Each launcher starts a fleet whose agents each hold a memory-bearing child, then is torn down exactly as it normally would be (raw: kill the roots; `tmux`: `kill-server`; MemMux: terminate each task). Ten seconds later we count survivors from the owned set (Table 3). The raw baseline strands every child it spawned—its teardown kills only the roots, orphaning the grandchildren—and the leak grows with the fleet, to ten processes and 303 MiB at N=10. `tmux` and MemMux both reap their full subtree at every size.

The honest reading: on *explicit* teardown, cleanup separates MemMux from the raw baseline, not from a real multiplexer—`tmux` reaps its subtrees too. This is the kind of result the claims discipline exists to surface.

5.3 H3: escaped-process visibility

We inject escapes: a stub forks an intermediate that spawns a memory-holding grandchild and then exits, reparenting the grandchild to `init` while it stays alive. The harness independently confirms the reparenting, then asks each launcher what it detected (Table 4).

Launcher	Fleet N	Leaked procs	Leaked MiB	Reclaimed
raw baseline	1	1	30.6	50.0%
raw baseline	5	5	151.8	50.0%
raw baseline	10	10	303.0	50.0%
`tmux` 3.4	10	0	0.0	100.0%
MemMux 0.8.0	10	0	0.0	100.0%

Table 3: Survivors after normal teardown (*hold*). The raw baseline leaks half of every fleet, growing to 303 MiB at N=10; `tmux` and MemMux reclaim 100% at all N.

Launcher	Injected	Detected	Note
raw-baseline	10	—	unsupported
tmux 3.4	10	—	unsupported
MemMux 0.8.0	1 / 5 / 10	1 / 5 / 10	detected all, every N

Table 4: Escaped-process detection (*escape*). "—" denotes no detection mechanism; it is never scored as zero-of-measured.

MemMux surfaces every escape—1/1, 5/5, 10/10—through events read from the daemon; the baselines have no mechanism to notice. This is a capability, not a tuning difference, and it is the cleanest separation from a real competitor.

5.4 H1 and H5: attribution and the cost of governance

Across every run MemMux attributed 100% of each launched tree's proportional memory to its owning task (H1). The cost of governance in *memory* is small and we report it plainly: the MemMux daemon's own resident footprint is 4.6–5.4 MiB (growing slightly with the fleet), against 2.4 MiB for the `tmux` server and zero for the raw baseline. The cost in *CPU* is the honest weak spot (H5): the always-on 1 Hz full-tree PSS scan costs 0.6% CPU at a single agent, 1.5% at five, and 2.7% at ten—over our 2% target once the fleet reaches ten. The scan is O(processes) and currently re-reads the whole tree each tick; incremental sampling that touches only changed pids is the obvious fix and is future work. We state the miss rather than bury it.

5.5 Validation on real coding agents

To test whether the attribution and overhead results are artifacts of the stub, we ran the *same* harness against a real coding agent. Each "agent" is a headless Claude Code session (`claude -p`, v2.1.170) editing a scratch Git repository, launched through the identical sampling and teardown pipeline via the benchmark's `--agent-cmd` mode. We report an N=3 hold run on the macOS host (Apple M4 Max, 36 GiB). A live Claude Code session is a multi-process, Node-based tree, so this is a strictly harder attribution target than the single-process stub.

Table 5 gives the result. MemMux attributes 100% of every real agent tree to its owning task—the H1 gate holds on live agents, not just the stub—and the manager cost stays small (7.2 MiB resident, 0.012% CPU at three agents, consistent with the small-fleet end of the H5 curve). Three concurrent real sessions occupy ∼0.85 GB (∼290 MiB each), confirming that the pressure the system governs is real and per-agent-substantial. We are deliberate about what this run does *not* show: because these sessions ran to completion and exited cleanly, every launcher (including the raw baseline) reclaimed 100% of its tree, so the cleanup and escaped-process *differentiators* do not appear here—they require the forced-termination and double-fork conditions that the controlled H2 and H3 scenarios above isolate. This experiment validates that attribution and overhead generalize from the stub to real, multi-process agent trees; the controlled scenarios remain the setting in which the governance differentiators are exercised.

Table 5: Real Claude Code agents (N=3, hold scenario, macOS) driven through the identical harness. Every launcher attributes the full real (Node-based) agent tree; MemMux's manager overhead is small. Footprint is steady-state provider+manager memory; at N=3 without an overcommit budget the totals are comparable across launchers—this run isolates *attribution and overhead on real trees*, not the overcommit footprint differentiator of Table 2.

Launcher	Total steady (MiB)	Manager (MiB)	Tree attributed
raw baseline	883.6	0.0	100%
`tmux` 3.6a	883.6	2.4	100%
`herdr` 0.8.0	929.6	18.2	100%
MemMux 0.8.0	843.5	7.2	100%

6 Discussion and Limitations

Synthetic workload. The largest threat to validity: agents are a deterministic memory-ballast stub, not live models. We trade realism for reproducibility and for identical workloads across launchers, which is what makes the comparison fair. The *qualitative* guarantees—attribution, cleanup, escape visibility, admission—do not depend on the workload, but the specific megabytes for a real Claude- or GPT-driven session will differ. We take a first step in §5.5, running real Claude Code sessions through the same harness and confirming that 100% attribution and low overhead hold on live, multi-process agent trees; a larger live-agent study—more agents, longer sessions, and the forced-termination conditions that expose the cleanup differentiator—remains the most important next step.

One host, modest fleets. We evaluate a single 32 GB Linux host (with a macOS cross-check) and $N \leq 10$. The harness sweeps to N=20 and the reproducer emits the full curve; we simply did not run larger fleets, on which the CPU-overhead question (H5) becomes sharper.

Conservative admission. MemMux reserves each task's class-predicted peak, which for our stub is well above its actual RSS—so the flat 435 MiB in Table 2 reflects admission protecting against the *reservation*, and MemMux leaves headroom on the table (it deferred agents that would have fit). An exponentially-weighted predictor that tightens reservations toward observed peaks is designed and is future work. The bounded-footprint claim stands; the specific admit count would rise with a tighter predictor.

Overhead not yet met; empirical, not formal. Monitoring exceeds 2% CPU by ten agents (above), and these are runtime monitors with measured pass/fail behavior, not machine-checked proofs. We claim the signals are checkable and checked, not that the engine is formally verified.

7 Reproducibility

MemMux and its benchmark are open source (MIT).[1] A single command runs the full matrix—every launcher, scenario, and fleet size, with repeated trials—captures the host specification, and regenerates every table and figure here plus the raw per-sample and per-process time series. Competitor versions and the measurement date are embedded in the generated report, and any launcher that could not be driven is listed with its reason. Reproducing the evaluation on another host is one invocation. The same harness accepts a `--agent-cmd` flag that substitutes a real coding-agent command (e.g. `claude -p ...`) for the stub, so the real-agent validation of §5.5 is reproducible through the identical pipeline; that run's report and raw per-process series are archived under `paper/realagent-run/`.

```
memmux-bench paper --out paper-out              # full matrix
memmux-bench run  --agents 3 --agent-cmd \
```

[1] Code, benchmark, and a one-command reproducer are open-source and will be released; the repository URL is withheld to preserve double-blind review.

```
    'claude -p "..."' --out real-out          # same harness, real agent
```

Each run writes `host.json` (OS, CPU model, core count, physical RAM, tool versions), a Markdown report, and per-cell `*.jsonl` / `*.procs.jsonl` time series (RSS, PSS, USS, swap, and page faults per process on Linux), plus the figures.

8 Conclusion

Fleets of coding agents have outrun the tools we use to run them: multiplexers arrange them, but nothing governs them. We argued that memory governance for such fleets is a verification problem and built MemMux to make five guarantees checkable at runtime, with a benchmark honest enough to admit where those guarantees do *not* distinguish us. On a 32 GB Linux host the payoff is concrete—a flat versus linear footprint under overcommit (4.9× at ten agents), complete cleanup where the raw baseline strands half a fleet, and full visibility into escaped processes that the alternatives cannot see—alongside an honest overhead cost we have not yet driven under 2%. The path from here is a live-agent workload and larger fleets; the reproducer makes both a single command away.

References

Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to runtime verification. In *Lectures on Runtime Verification: Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 1–33. Springer, 2018. doi: 10.1007/978-3-319-75632-5_1.

Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. doi: 10.1145/2890784.

Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for fine-grained resource sharing in the data center. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2011.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

Linux Kernel Documentation. Control group v2. https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html, 2024a. Accessed 2026-08.

Linux Kernel Documentation. Out of memory management. https://www.kernel.org/doc/html/latest/admin-guide/mm/concepts.html, 2024b. Accessed 2026-08.

Linux Kernel Documentation. The /proc filesystem: smaps and smaps_rollup (proportional set size). https://www.kernel.org/doc/html/latest/filesystems/proc.html, 2024c. Accessed 2026-08.

Nicholas Marriott and tmux contributors. tmux: a terminal multiplexer. https://github.com/tmux/tmux, 2024. Accessed 2026-08.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. arXiv:2304.03442.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.

Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*, 2015. doi: 10.1145/2741948.2741964.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.